

Inference Engineering

Assignment

1. Difference between Latency and Throughput

Latency and throughput are two different ways of measuring the performance of a model during inference. They both tell us how well a model is performing, but they focus on different things.

Latency is the amount of time taken by the model to process one request and produce an output. For example, if a model takes 200 milliseconds to generate a response, its latency is 200 milliseconds. Lower latency is useful when we want the model to respond quickly to individual users.

Throughput is the amount of data or number of requests that the model can process in a given amount of time. It can be measured in requests per second or tokens per second. Higher throughput means that the system can handle more requests in the same amount of time.

Which one affects performance more depends on the application. For an interactive chatbot, latency is usually more important because the user wants a quick response. On the other hand, for a server handling many users at the same time, throughput can be more important. Factors such as batch size, model size, GPU hardware, memory bandwidth, and the type of workload can affect both latency and throughput.

2. KV Cache

KV cache is a technique used in language models during text generation to avoid repeating the same computations for tokens that have already been processed.

In the attention mechanism, the model creates Key (K) and Value (V) tensors for the input tokens. When the model generates the next token, the Keys and Values of the previous tokens do not need to be calculated again. Instead, they can be stored in memory and reused for the next generation step.

For example, when a model is generating a sentence one token at a time, it stores the Key and Value information from the tokens that have already been processed. When a new token is generated, the model only needs to calculate the new information and can use the previously stored values from the KV cache.

Without a KV cache, the model would have to repeatedly calculate the Key and Value information for all previous tokens at every generation step. This would require more computation as the sequence becomes longer. Therefore, KV cache helps reduce repeated computation and makes autoregressive text generation faster.

3. Quantization and Accuracy Trade-off

Quantization is the process of representing model values using lower precision data types. Instead of storing values using FP32, which uses 32 bits for each value, we can use formats such as INT8 or INT4. This reduces the memory required to store the model and can also make inference faster on hardware that supports these lower precision operations.

The basic idea is to map a range of floating-point values to a smaller integer range. One common method is affine quantization. The scale can be calculated using:

$$\text{scale} = \frac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}}$$

The zero-point can be calculated using:

$$\text{zero_point} = \text{round} \left(q_{\min} - \frac{x_{\min}}{\text{scale}} \right)$$

After finding the scale and zero-point, a floating-point value can be converted into a quantized value using:

$$q = \text{round} \left(\frac{x}{\text{scale}} + \text{zero_point} \right)$$

For example, suppose we have the floating-point values:

$$[0.0, 1.0, 2.0, 3.0]$$

and we want to convert them to INT8. The INT8 range is from -128 to 127 , so:

$$q_{\min} = -128, \quad q_{\max} = 127$$

The scale is:

$$\text{scale} = \frac{3.0 - 0.0}{127 - (-128)} = \frac{3}{255} \approx 0.01176$$

The zero-point is:

$$\text{zero_point} = \text{round} \left(-128 - \frac{0}{0.01176} \right) = -128$$

For example, for $x = 1.0$:

$$q = \text{round} \left(\frac{1.0}{0.01176} - 128 \right) \approx -43$$

Therefore, the value 1.0 can be represented approximately by the INT8 value -43 along with the scale and zero-point.

The main advantage of quantization is that it reduces the memory required by the model and can improve inference speed. However, using fewer bits also means that the original values cannot be represented as precisely. This can result in some loss of model accuracy. INT8 usually provides a good balance between memory reduction, speed, and accuracy, while INT4 gives even greater memory savings but can cause a larger loss in accuracy depending on the model and the quantization method.

4. Inference Serving Framework

One example of a real inference serving framework is **vLLM**. It is designed for efficiently serving large language models and handling multiple requests.

One of the important optimizations used by vLLM is PagedAttention. It manages the KV cache in smaller blocks instead of requiring one large continuous memory area for every request.

This makes memory usage more efficient and allows the system to handle more requests at the same time. Because of this, vLLM can achieve higher throughput when serving large language models while still providing good inference performance.